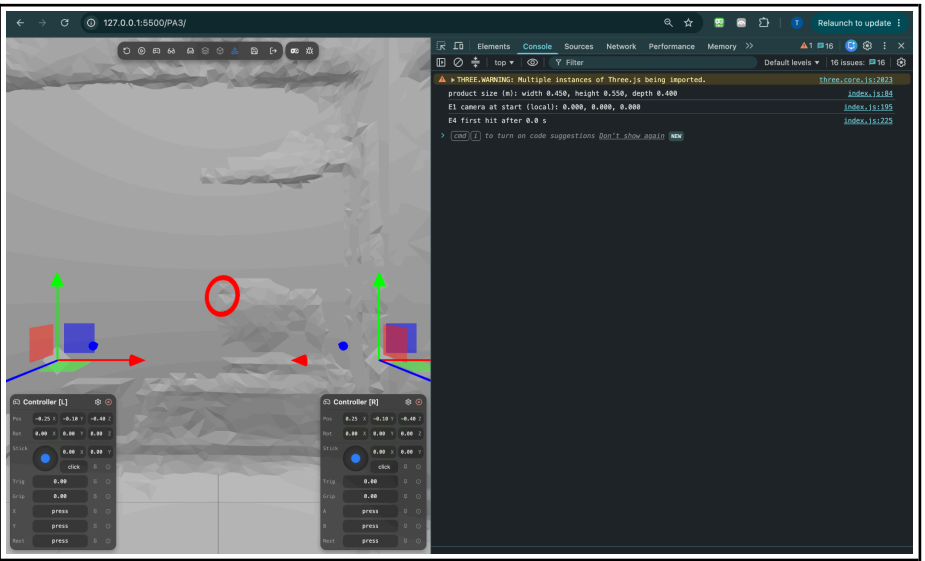
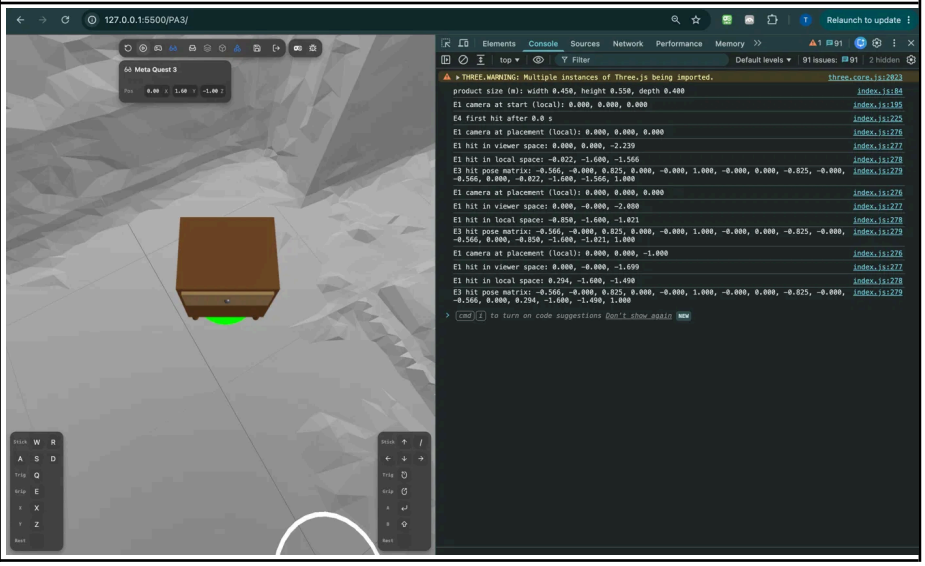
Write-up

Student: Margarita Serebrennikova
Student ID: 240137
VARIANT = "bedside table, single, 12°, faces user";
PLACEMENT_MODE = "single";
TOLERANCE_DEG = 12;

E1 — Reference spaces

Prediction: at session start the camera is near (0, 0, 0) in local space. After walking 1 m forward, z drops by about 1. A floor hit is roughly (0, 0, −d) in viewer space and has y ≈ −(phone height at start) in local space.

Camera start scene	
at z=-1.00	

Logged values	*x*	*y*	*z*
Camera at start (local)	0.000	0.000	0.000
Camera after 1m move (local)	0.000	0.000	-1.000
Hit in viewer space	0.000	0.000	-1.699
Hit in local space	0.294	-1.600	-1.490

Distance from the camera (0, 0, −1) to the local hit (0.294, −1.6, −1.49):

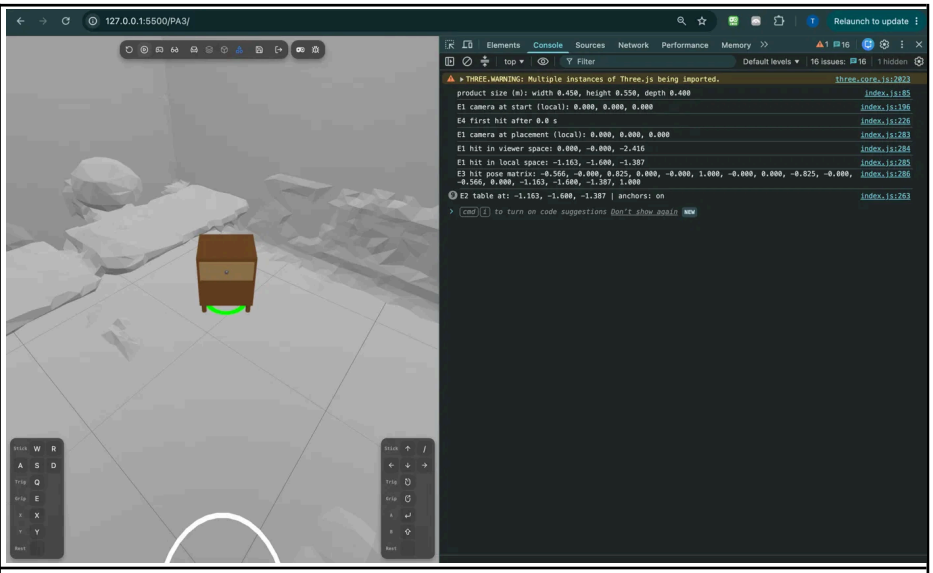
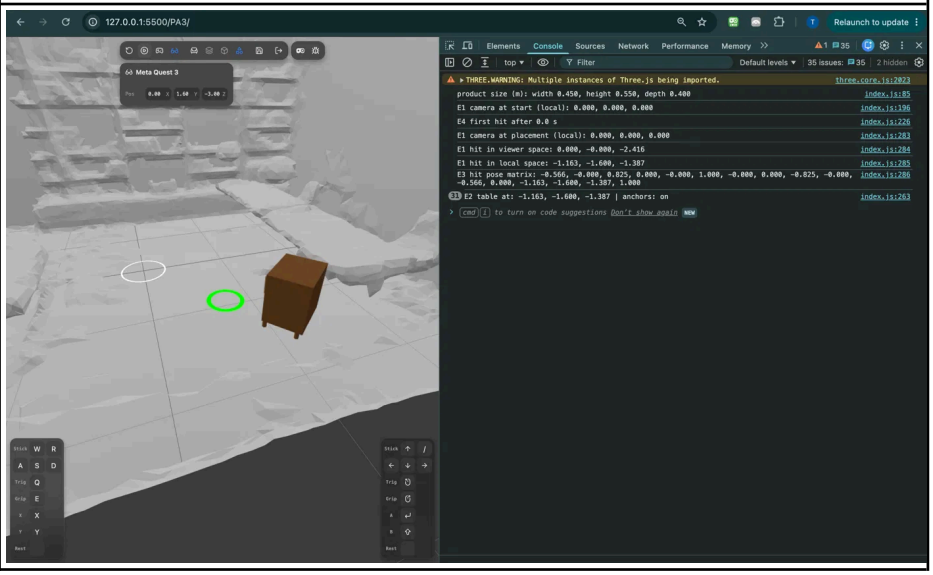
$$\sqrt{(0.294^2 + 1.6^2 + 0.49^2)} = \sqrt{2.886} = 1.699\,m$$

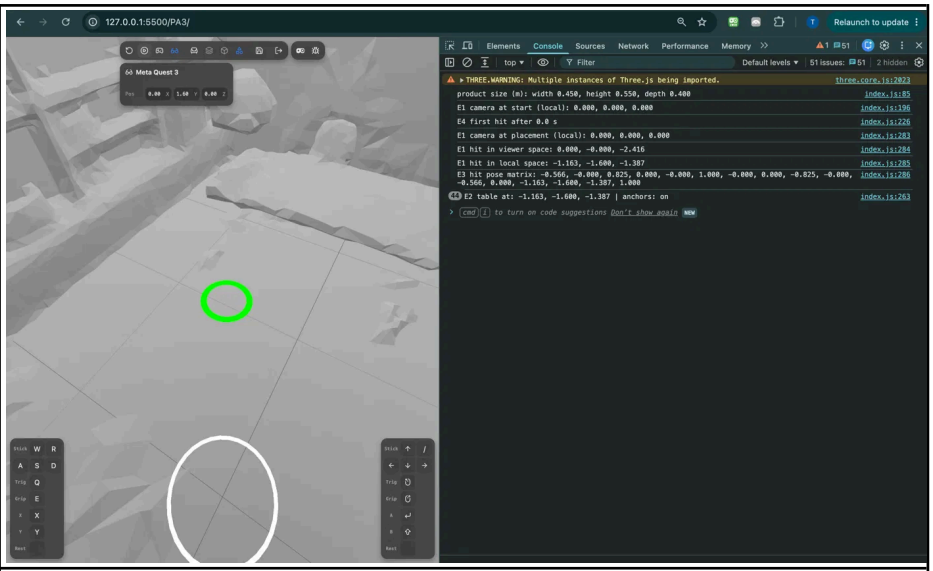
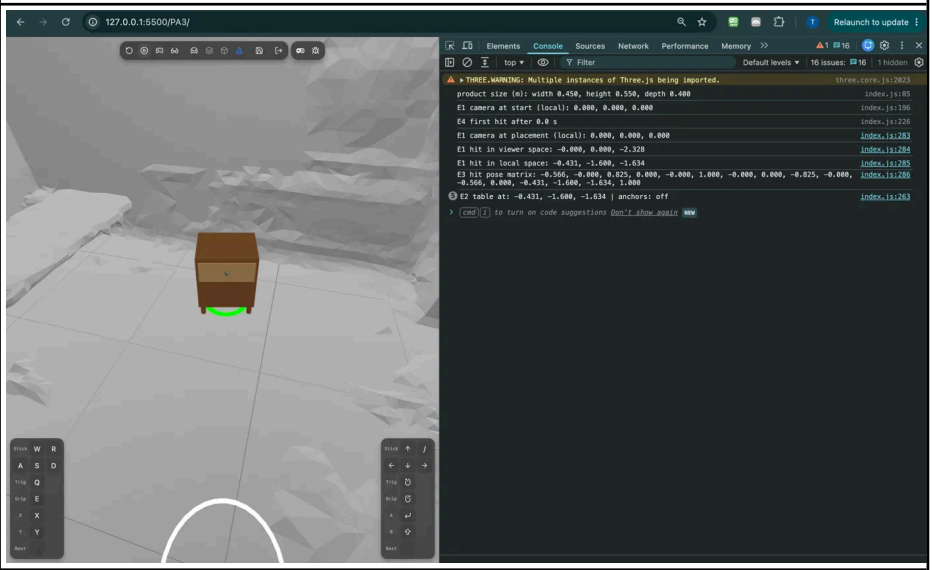
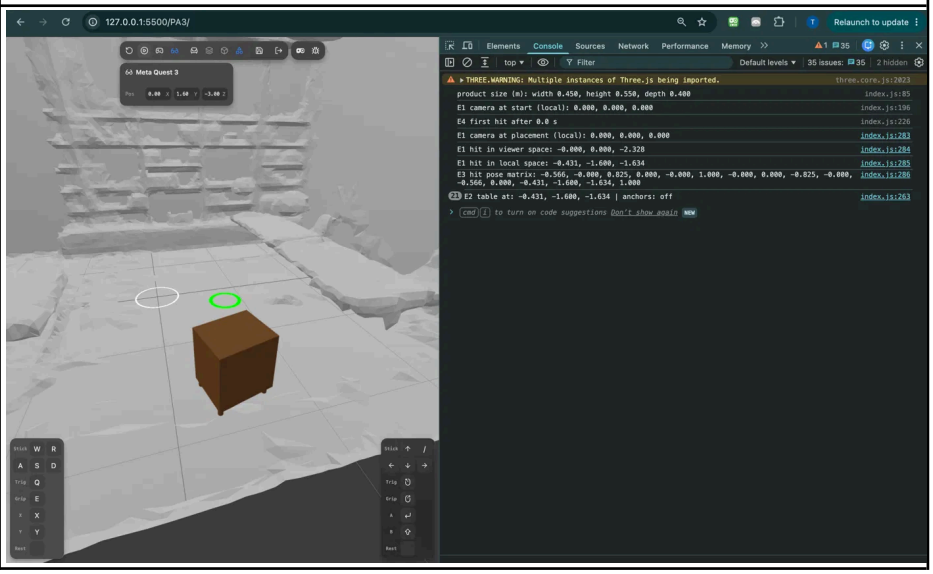
Local is fixed to the world. Its origin is the headset's pose when the session started, so floor points keep their coordinates as I move. Viewer is attached to the device, so the same floor point is described relative to it and changes with every movement. The two poses are one physical point seen from two origins. The hit-test source is created from viewer because the ray must start at the screen centre and follow the device. Results are read in local because the table needs world-fixed coordinates; in viewer space it would travel with the device.

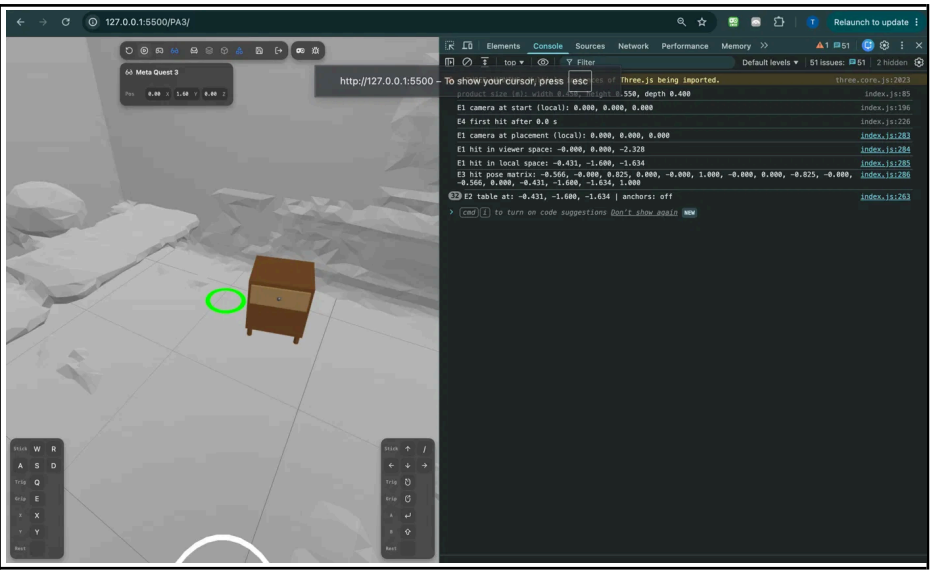
E2 — Checking Chapter 7

(a) Anchors on vs off

Prediction: with anchors the table stays on its floor mark after a 3 m walk; without them it drifts.

Anchors ON z=0, t=0	
Anchors ON, z=-3	

Anchors ON, getting z back to 0	
Anchors OFF, z=0.0, t=0	
Anchors OFF, z=-3.0	

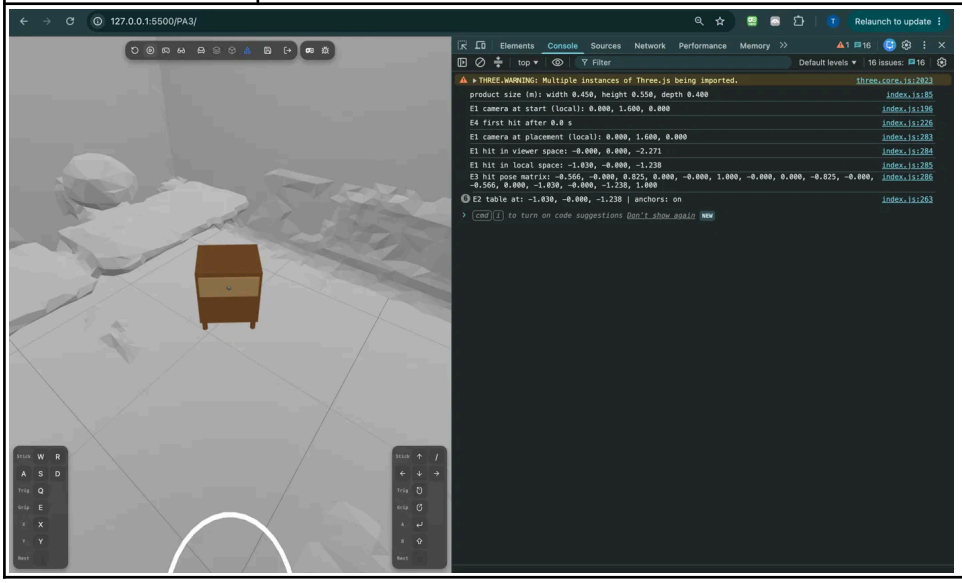
Anchors OFF, z back to 0	

Mode	*Position before*	*Position after 3m and back*	*Drift*
Anchor ON	(-1.163, -1.600, -1.387)	(-1.163, -1.600, -1.387)	0.0cm
Anchor OFF	(-0.431, -1.600, -1.634)	(-0.431, -1.600, -1.634)	0.0cm

Result: no drift in either mode. Emulator's tracking is simulated and exact, so there was no map correction for an anchor to apply. On a phone, VIO errors shift the map, and only an anchored product follows that correction. Ch. 7 is wrong that getHitTestResults() saves each hit with the Spatial Anchors module. A hit result is a one-time pose, valid only in its frame. An anchor is a point the device keeps tracking: its anchorSpace is updated as the map improves, and my code re-reads frame.getPose(anchor.anchorSpace, localSpace) every frame.

(b) local-floor

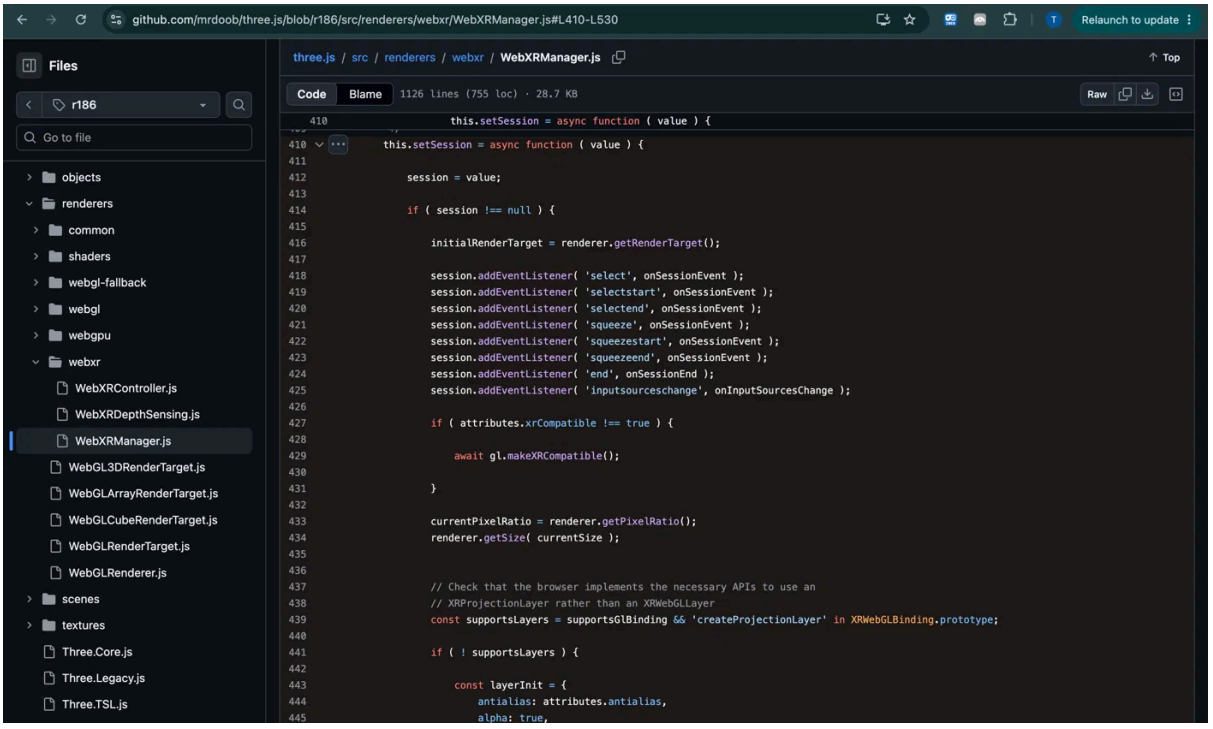
Prediction: the origin moves down to the floor, so the camera starts at y ≈ 1.6 and the floor hit at y ≈ 0.

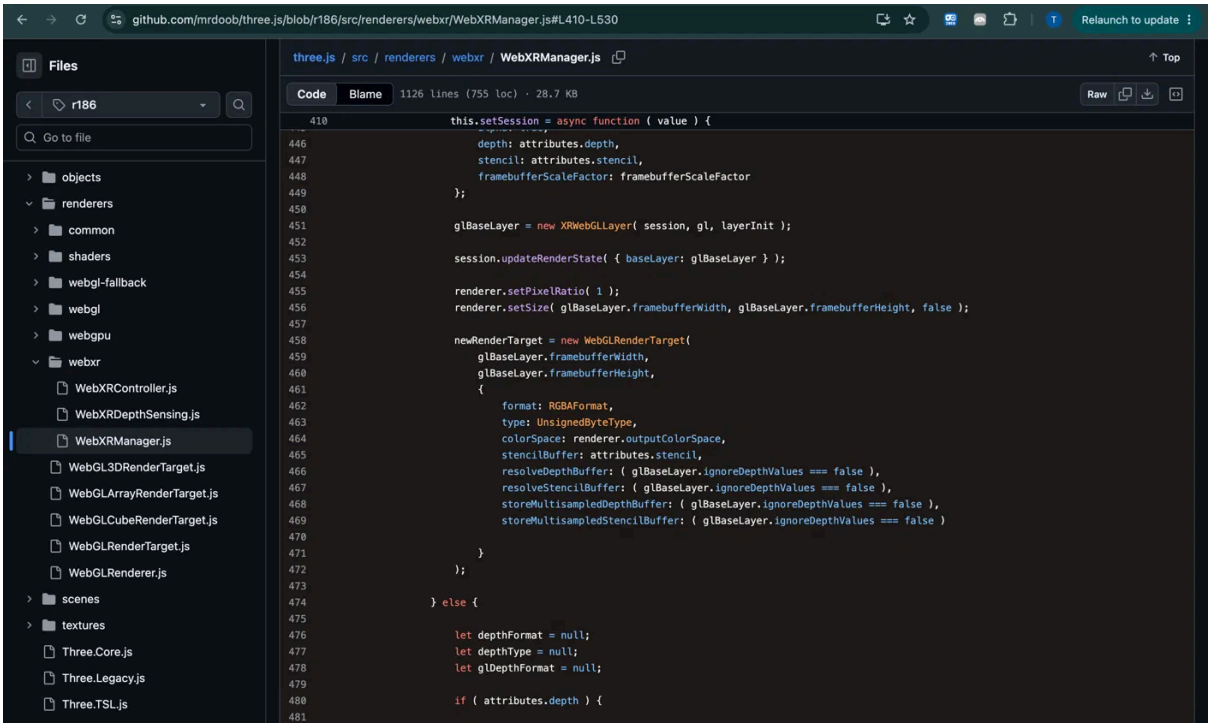
Result	
	

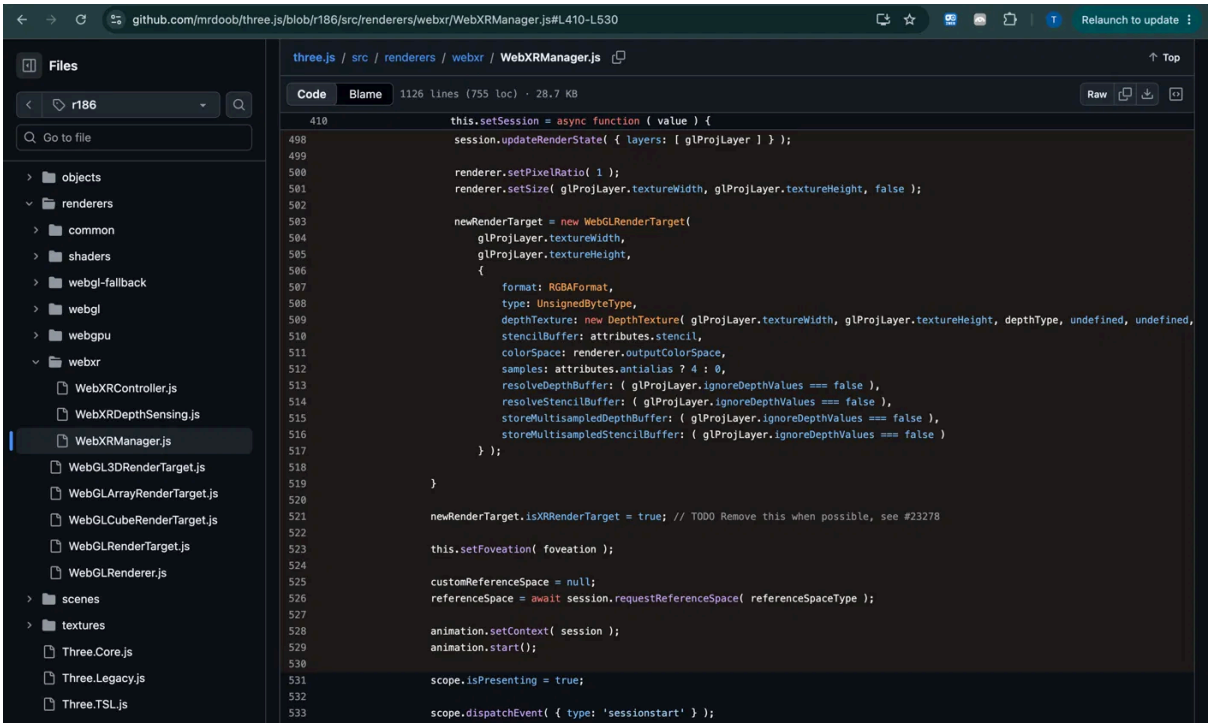
Result: every y value shifted by exactly +1.600 m, the estimated floor height. The hit was still found at once on the same floor, with the same 0° normal. local-floor only moves the origin to

the floor; it does not change how surfaces are detected, so it does not help hit testing, contrary to Ch. 7.

(c) setSession in WebXRManager.js (0.186.1)



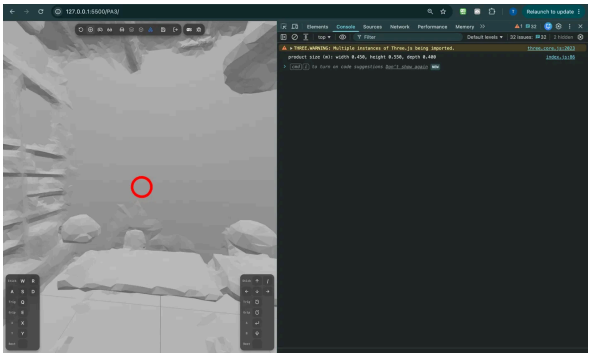


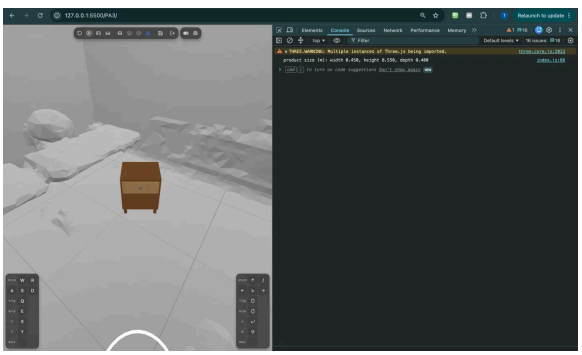


setSession already does the work for book's setupWebGLLayer(): line 429 makes the context XR-compatible and lines 451-453 create and attach the layer. On browsers with WebXR Layers it uses a projection layer instead (line 498). So setupWebGLLayer() is no longer needed; my code passes xrCompatible: true to getContext and calls only setReferenceSpaceType and setSession.

E3 — Reading the pose matrix

Prediction: on the floor normal points almost straight up, so element m5 ≈ 1 and the tilt is near 0°.





The matrix is column-major; each group of four is one column:

Column	*Elements*	*Meaning*	*My values*
0	m0-m3	x-axis	-0.566, 0.000, 0.825, 0.000
1	m4-m7	y-axis - surface normal	0.000, 1.000, 0.000, 0.000
2	m8-m11	z-axis	-0.825, 0.000, -0.566, 0.000
3	m12-m15	position (x,y,z,1)	-1.163, -1.600, -1.387, 1.000

Tilt by hand: cos θ = (m4, m5, m6) · (0, 1, 0) = m5 = 1.000, so θ = arccos(1.000) = 0.0°. That is ≤ 12°, so the reticle is green and placement is allowed.

Explanation: The rotation columns of a pose are its own axes expressed in world space, so the y column is the surface's "up". Its y component equals the cosine of the angle from world up, which is why my filter only needs acos(m5). The emulator has no tilted board, so I used a wall for the red case: its normal is nearly horizontal, which gives a large angle, and the tap placed nothing.

E4 — How the phone finds surfaces

Camera finds feature points: corners and texture details that are easy to recognise again. Visual-inertial odometry tracks those points from frame to frame and fuses them with the gyroscope and accelerometer, so the phone estimates its own motion. Points that lie on one flat surface are fitted to a plane, and the hit test intersects the screen-centre ray with those planes.

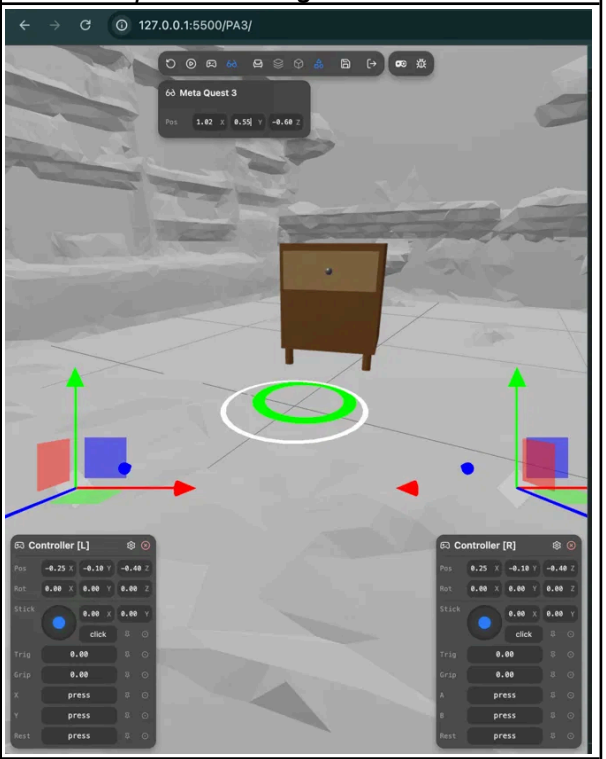
Surface	*Seconds to first hit*	*Success*
Emulator floor	0.0	yes
Emulator rock wall	0.0	yes
Patterned carpet	not tested - not ARC core device	

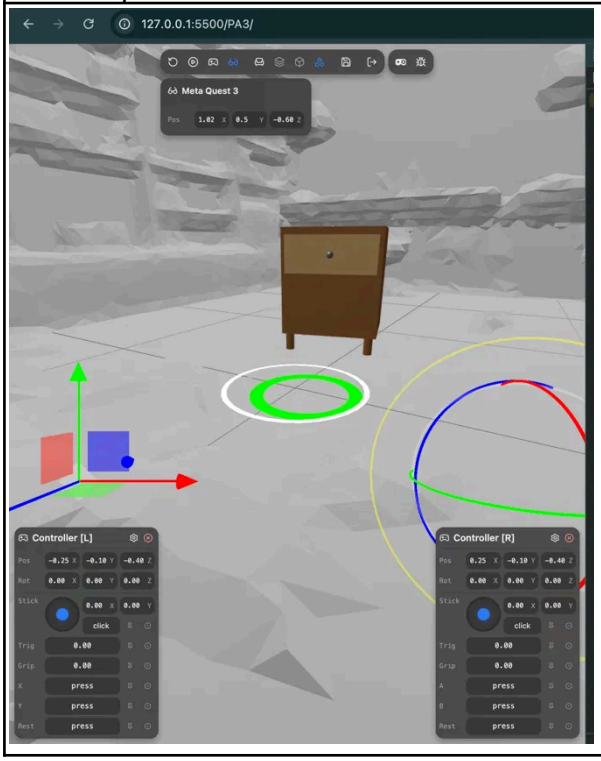
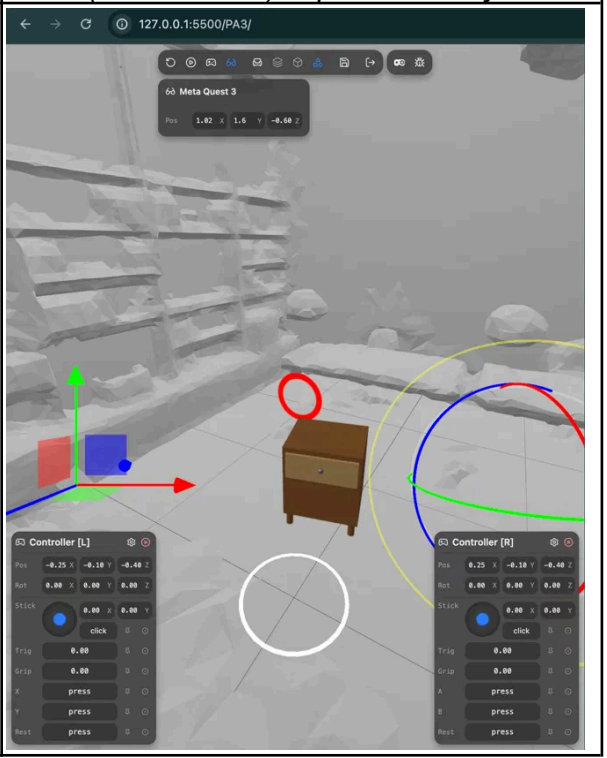
E5 — Scale check

Prediction: The table should measure 55 cm, because Box3 logged width 0.450, height 0.550, depth 0.400 m. So the top surface should turn edge-on when the headset is at Pos Y = 0.55 m, be visible above that and hidden below it.

Result.

The emulator room has no real object of known size, so I used the headset height (Pos Y, measured from the emulator floor) as the ruler. Horizontal top surface is visible only from above, edge-on at its own height, and hidden from below.

Y=0.6: top surface thin strip visible	*Y=0.55: top surface edge-on*
	

Y=0.5: top surface hidden	*Y=1.6 (back to initial): top surface fully visible*
	

Two likely error sources:

- Resolution of the method. I only tested Pos Y in steps of 5 cm (0.60, 0.55, 0.50), so the true edge-on height could be anywhere within about ±2.5 cm. On a phone, the detected floor plane can also sit a few cm above or below the real floor, which shifts the whole table.
- Judging “edge-on” by eye. With perspective, a very thin strip of the top is hard to see on screen, so the switch point is judged visually. On a phone, VIO tracking-scale errors would add to this.

E6 — Product preview for a real retailer

Two failure modes from my tests would cost a Kazakhstani furniture retailer a sale.

1. on any surface tilted more than 12° the reticle turned red and taps did nothing, with no explanation; a customer on an uneven rug would think the page is broken.
   - **Fix**: show “surface too tilted - aim at flat floor” next to the red reticle.

2. my desktop browser only showed the fallback message, and iPhone Safari has no immersive-ar, so many customers could not preview at all.
   - **Fix**: offer a plain 3D viewer with the exact dimensions as a fallback.

WebXR page can read the device pose, hit-test points with surface orientation, and anchors. It cannot read camera pixels unless it requests camera access, which this page does not. Users should be told the camera image stays on their phone and only surface positions are used.

Development log:

- **Bug**: the page showed an empty status box and no START AR button; an extra) after the .catch() in onRequestSession stopped the whole module from running.
- How found: the DevTools Console showed SyntaxError: Unexpected token ')' at index.js:154.
- Fix: removed the extra bracket; the overlay and the fallback message appeared.